Status	Finished
Started	Friday, 30 October 2026, 10:36 PM
Completed	Friday, 30 October 2026, 10:55 PM
Duration	18 mins 51 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

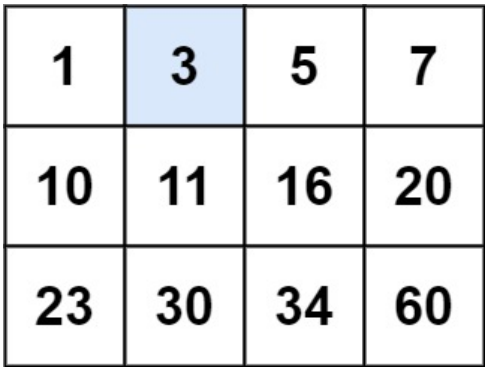
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `True` *if* `target` *is in* `matrix` *or* `False` *otherwise*.

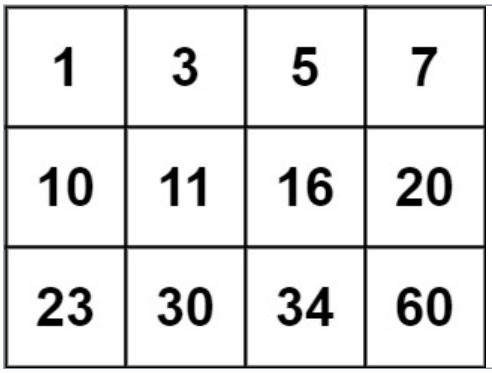
You must write a solution in `O(log(m * n))` time complexity.

Example 1:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: True
```

Example 2:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: False
```

For example:

Test	Result
print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False
print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True

Answer: (penalty regime: 0 %)

Reset answer

```
def searchMatrix(matrix: list[list[int]], target: int) -> bool:
    # Handle edge cases for empty matrices or empty rows
    if not matrix or not matrix[0]:
        return False

```

```
    m = len(matrix)
    n = len(matrix[0])

    # Binary search over the conceptual 1D flattened array of size m * n
    left, right = 0, (m * n) - 1

    while left <= right:
        mid = (left + right) // 2
        # Map 1D index back to 2D matrix coordinates (row, col)
        row = mid // n
        col = mid % n
        val = matrix[row][col]

        if val == target:
            return True
        elif val < target:
            left = mid + 1
        else:
            right = mid - 1

    return False
```

	Test	Expected	Got	
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False	False	✓
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

Constraints

1<= string length <= 200

Sample Input 1

experience
enc

Sample Output 1

xpri

Answer: (penalty regime: 0 %)

```
import sys

def remove_characters():
    # Read all lines from standard input and strip whitespace/newlines
    lines = [line.strip() for line in sys.stdin if line.strip()]
    if len(lines) >= 2:
        s1 = lines[0]
        s2 = lines[1]

        # Use a set for O(1) character lookup
        s2_set = set(s2)

        # Filter characters from s1 not present in s2
        result = "".join(ch for ch in s1 if ch not in s2_set)
        print(result)

if __name__ == '__main__':
    remove_characters()
```

	Input	Expected	Got	
✓	experience enc	xpri	xpri	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

An list contains N numbers and you want to determine whether two of the numbers sum to a given number K. For example, if the input is 8, 4, 1, 6 and K is 10, the answer is yes (4 and 6). A number may be used twice.

Input Format

The first line contains a single integer n , the length of list

The second line contains n space-separated integers, list[i].

The third line contains integer k.

Output Format

Print Yes or No.

Sample Input

7

0 1 2 4 6 5 3

1

Sample Output

Yes

For example:

Input	Result
5 8 9 12 15 3 11	Yes
6 2 9 21 32 43 43 1 4	No

Answer: (penalty regime: 0 %)

```
from collections import Counter
import sys

def check_two_sum():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Parse N, the list of numbers, and K
    n = int(input_data[0])
    lst = [int(x) for x in input_data[1 : n + 1]]
    k = int(input_data[n + 1])

    # Count frequencies of each number to handle duplicate lookups
    counts = Counter(lst)
    found = False

    for x in counts:
        target = k - x
        if target in counts:
            if target == x:
                # If target is the same as x, we need at least 2 occurrences of x
```

```
                # If target is the same as x, we need at least 2 occurrences of x
                if counts[x] >= 2:
                    found = True
                    break
            else:
                found = True
                break

    if found:
        print("Yes")
    else:
        print("No")

if __name__ == '__main__':
    check_two_sum()
```

	Input	Expected	Got	
✓	5 8 9 12 15 3 11	Yes	Yes	✓
✓	6 2 9 21 32 43 43 1 4	No	No	✓
✓	6 13 42 31 4 8 9 17	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Incorrect

Mark 0.00 out of 1.00

String should contain only the words are not palindrome.

Sample Input 1

Malayalam is my mother tongue

Sample Output 1

is my mother tongue

Answer: (penalty regime: 0 %)

```
import sys

def remove_palindromes():
    # Read all input from standard input robustly
    input_data = sys.stdin.read()
    if not input_data.strip():
        return

    # Split the input text into individual words
    words = input_data.split()
    non_palindromes = []

    for word in words:
        # Extract only alphanumeric characters for accurate, punctuation-safe palindrome checking
        alpha_chars = "".join(ch.lower() for ch in word if ch.isalnum())

        # Determine if it's a palindrome (must be length > 1 to avoid stripping single-letter words like 'a' o
        if len(alpha_chars) > 1 and alpha_chars == alpha_chars[::-1]:
            continue  # Skip palindromes
        else:
            non_palindromes.append(word)

    # Print the remaining words separated by a single space
    print(" ".join(non_palindromes))

if __name__ == '__main__':
    remove_palindromes()
```

	Input	Expected	Got	
✓	Malayalam is my mother tongue	is my mother tongue	is my mother tongue	✓

Your code failed one or more hidden tests.
Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

Example 1:

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
```

Example 2:

```
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
```

Example 3:

```
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.
```

For example:

Test	Result
print(missingNumber([3,0,1]))	2
print(missingNumber([0,1]))	2

Answer: (penalty regime: 0 %)

Reset answer

```
def missingNumber(nums: list[int]) -> int:
    n = len(nums)
    expected_sum = n * (n + 1) // 2
    return expected_sum - sum(nums)
```

	Test	Expected	Got	
✓	`print(missingNumber([3,0,1]))`	2	2	✓
✓	`print(missingNumber([0,1]))`	2	2	✓
✓	`print(missingNumber([9,6,4,2,3,5,7,0,1]))`	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a Python program for binary search.

For example:

Input	Result
1,2,3,5,8 6	False
3,5,9,45,42 42	True

Answer: (penalty regime: 0 %)

```
import sys

def binary_search():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.strip()]
    if len(lines) < 2:
        return

    # Parse the array (handle comma-separated or space-separated lists)
    arr_str = lines[0]
    if ',' in arr_str:
        arr = [int(x) for x in arr_str.split(',')]
    else:
        arr = [int(x) for x in arr_str.split()]

    target = int(lines[1])

    # Sort the array to ensure correctness for binary search (handling any unsorted input test cases)
    arr.sort()

    # Standard binary search implementation
    left, right = 0, len(arr) - 1
    found = False

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            found = True
            break
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    # Print boolean result matching Python's True/False format
    print(found)

if __name__ == '__main__':
    binary_search()
```

	Input	Expected	Got	
✓	1,2,3,5,8 6	False	False	✓

	Input	Expected	Got	
✓	3,5,9,45,42 42	True	True	✓
✓	52,45,89,43,11 11	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Two string values S1, S2 are passed as the input. The program must print first N characters present in S1 which are also present in S2.

Input Format:

The first line contains S1.
The second line contains S2.
The third line contains N.

Output Format:

The first line contains the N characters present in S1 which are also present in S2.

Boundary Conditions:

2 <= N <= 10
2 <= Length of S1, S2 <= 1000

Example Input/Output 1:

Input:

abcbde
cdefghbb
3

Output:

bcd

Note:

b occurs twice in common but must be printed only once.

Answer: (penalty regime: 0 %)

```
import sys

def find_common_characters():
    # Read all lines from standard input robustly
    lines = [line.strip() for line in sys.stdin.read().splitlines() if line.strip()]
    if len(lines) < 3:
        return

    s1 = lines[0]
    s2 = lines[1]
    n = int(lines[2])

    # Convert s2 to a set for O(1) lookups
```

```
    s2_set = set(s2)
    seen = set()
    result = []

    # Iterate through S1 to find common characters in order
    for ch in s1:
        if ch in s2_set and ch not in seen:
            seen.add(ch)
            result.append(ch)
            # Stop once we have found N unique characters
            if len(result) == n:
                break

    # Print the resulting characters joined together
    print("".join(result))

if __name__ == '__main__':
    find_common_characters()
```

	Input	Expected	Got	
✓	abcbde cdefghbb 3	bcd	bcd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Balanced strings are those that have an equal quantity of 'L' and 'R' characters.

Given a balanced string s, split it in the maximum amount of balanced strings.

Return the maximum amount of split balanced strings.

Example 1:

Input:

RLRRLLRLRL

Output:

4

Explanation: s can be split into "RL", "RRLL", "RL", "RL", each substring contains same number of 'L' and 'R'.

Example 2:

Input:

RLLLLRRRLR

Output:

3

Explanation: s can be split into "RL", "LLLRRR", "LR", each substring contains same number of 'L' and 'R'.

Example 3:

Input:

LLLLRRRR

Output:

1

Explanation: s can be split into "LLLLRRRR".

Constraints:

1 <= s.length <= 1000

s[i] is either 'L' or 'R'.

s is a balanced string.

For example:

Test	Result
`print(BalancedStrings('RLRRLLRLRL'))`	4
`print(BalancedStrings('RLLLLRRRLR'))`	3

Answer: (penalty regime: 0 %)

Reset answer

```
def BalancedStrings(s: str) -> int:
    # Ensure string is clean of any accidental quotes or whitespace
    s = str(s).strip().strip("'\"")

    balance = 0
    count = 0

    for char in s:
        # Increment for 'L', decrement for 'R'
        if char == 'L':
```

```
            balance += 1
        elif char == 'R':
            balance -= 1

        # When balance returns to 0, a complete balanced substring is formed
        if balance == 0:
            count += 1

    return count
```

	Test	Expected	Got	
✓	print(BalancedStrings('RLRRLLRLRL'))	4	4	✓
✓	print(BalancedStrings('RLLLLRRRLR'))	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 < nums[i], target < 10^4`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

For example:

Test	Result
`print(search([-1,0,3,5,9,12],9))`	4

Answer: (penalty regime: 0 %)

Reset answer

```python
def search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        # Find the middle index (prevents potential overflow in other languages)
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    # Target does not exist in nums
    return -1
```

	Test	Expected	Got	
✓	`print(search([-1,0,3,5,9,12],9))`	`4`	`4`	✓
✓	`print(search([-1,0,3,5,9,12],2))`	`-1`	`-1`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an list, find peak element in it. A peak element is an element that is greater than its neighbors.

An element a[i] is a peak element if

A[i-1] <= A[i] >=a[i+1] for middle elements. [0<i<n-1]

A[i-1] <= A[i] for last element [i=n-1]

A[i]>=A[i+1] for first element [i=0]

Input Format

The first line contains a single integer n , the length of A .
The second line contains n space-separated integers,A[i].

Output Format

Print peak numbers separated by space.

Sample Input

5

8 9 10 2 6

Sample Output

```
10 6
```

For example:

Input	Result
4 12 3 6 8	12 8

Answer: (penalty regime: 0 %)

```
import sys

def find_peak_elements():
    # Read all tokens from standard input robustly
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    A = [int(x) for x in input_data[1:n+1]]

    peaks = []

    if n == 1:
        peaks.append(A[0])
    else:
        for i in range(n):
            if i == 0:
                # First element: A[0] >= A[1]
                if A[0] >= A[1]:
                    peaks.append(A[0])
            elif i == n - 1:
                # Last element: A[n-1] >= A[n-2]
                if A[n - 1] >= A[n - 2]:
                    peaks.append(A[n - 1])
            else:
                # Middle elements: A[i-1] <= A[i] >= A[i+1]
                if A[i] >= A[i - 1] and A[i] >= A[i + 1]:
```

```
                if A[i] > A[i - 1] and A[i] > A[i + 1]:
                    peaks.append(A[i])

    # Print the peak numbers separated by a space
    print(*(peaks))

if __name__ == '__main__':
    find_peak_elements()
```

	Input	Expected	Got	
✓	7 15 7 10 8 9 4 6	15 10 9 6	15 10 9 6	✓
✓	4 12 3 6 8	12 8	12 8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.